



Interrupciones

Organización y Arquitectura de las Computadoras

Interrupciones



Una interrupción es generada por un CALL de la circuitería del sistema (derivada externamente de una señal – interrupción por **hardware**) o por una CALL generada en un programa (derivada internamente de una instrucción – interrupción por **software**).

Cualquiera interrumpirá el programa para llamar una subrutina de servicio de interrupción (RSI) o manejador de interrupción.

Las interrupciones generadas por programación las cuales son un tipo de instrucción CALL en el microprocesador 8088/86 se dividen en tres tipos de instrucciones (**INT**, **INTO**, e **INT3**).

INT n



Esta instrucción provoca una transición controlada a un servicio de interrupción específico, identificado por el número n. El procesador guarda automáticamente la dirección de retorno y cambia a modo privilegiado (Modo Kernel, el procesador cambia de nivel de privilegio, permite ejecutar código con acceso a recursos de sistema más elevados, como el kernel del sistema operativo) para ejecutar el servicio de interrupción. Esta instrucción se utiliza comúnmente para realizar llamadas al sistema en sistemas operativos.

Su tamaño es de un byte (del opcode) más el tamaño del hexadecimal que representa la interrupción, **INT 21H** es de 2 bytes por ejemplo.

INTO



Esta instrucción verifica la bandera de desbordamiento (**OF**) y genera una interrupción si está activada. Se utiliza comúnmente en operaciones aritméticas para manejar casos de desbordamiento.

Su tamaño es de un byte del opcode ya que no tiene variaciones y su finalidad es no ocupar mucho espacio para ser eficiente en su llamado, ejecución y retorno.

INT3



La instrucción INT3 se utiliza como una instrucción de ruptura o trampa. Su propósito principal es proporcionar un medio para la depuración del software. La instrucción INT3 (o 0xCC en hexadecimal) genera una interrupción de software de propósito general. Los depuradores pueden aprovechar esto para detener la ejecución del programa cuando se encuentra esta instrucción, permitiendo al programador examinar el estado del programa y depurarlo.

Su tamaño es de un byte del opcode ya que no tiene variaciones y su finalidad es no ocupar mucho espacio para ser eficiente en su llamado, ejecución y retorno.

Vector de Interrupciones



El vector de interrupciones es una tabla de direcciones que contiene las direcciones de las rutinas de servicio de interrupción. Cuando se produce una interrupción, el procesador utiliza un número de interrupción para indexar en esta tabla y encontrar la dirección de la rutina de servicio de interrupción correspondiente.

Un vector de interrupciones es un número de 4 bytes almacenado en los primeros 1024 bytes de memoria (00000H-003FFH).

Hay 256 vectores de interrupción diferentes ($1024/4$), cada uno de los cuales contiene la dirección de una subrutina de servicio de interrupción, la subrutina llamada por un interruptor.

Cada vector contiene un valor para **IP** y **CS** que forma la dirección de la subrutina de servicio de interrupción. Los primeros dos bytes contienen el IP y los últimos dos bytes contienen CS.

Vector de Interrupciones



TABLA 1. Vector de Interrupcion

Número	Dirección	Función
0	0H-3H	Division por cero
1	4H-7H	Paso sencillo
2	8H-BH	NMI (interrupcion por circuiteria)
3	CH-FH	Punto de ruptura o de paro (Breakpoint)
4	10H-13H	Interrupcion por sobre flujo
5-31	14H-7FH	Reservadas para futura utilizacion*
32-255	80H-3FFH	Interrupciones del usuario

Intel reservó los primeros 32 vectores de interrupción para el 80286 y futuros productos .
Los vectores de interrupciones restantes (32-255) están disponibles para el usuario.

Vectores de Interrupciones



El 8088/86 tiene disponibles tres instrucciones de interrupción diferentes para el programador: INT, INTO, e INT 3.

Cada una de esas instrucciones busca un vector en la tabla de vectores y luego llama a una subrutina almacenada en la localidad direccionada por el vector.

La llamada al interruptor es similar a una instrucción CALL lejana porque está coloca en la pila la dirección de retorno (IP y CS). La diferencia es que este también coloca en la pila una copia de registro de banderas.

Vectores de Interrupciones (INT XXh)



Hay 256 instrucciones de interrupción por programación diferentes (INT), disponibles para el programador. Cada instrucción INT tiene un operando numérico cuyo rango es entre 0 y 255 (00H-FFH).

Por ejemplo: Para una interrupción creada por el programador, INT 64h esta usa el vector de interrupciones 100, el cual aparece en la dirección de memoria 190H-193H.

Se calcula la dirección del vector de interrupción multiplicando el número del tipo de interrupción por 4, para el ejemplo fue $64H \times 4 = 190H$.

Por ejemplo; la instrucción INT 10H llama a la subrutina de servicio de interrupción cuya dirección está almacenada empezando en la localidad de memoria 40H de $(10H \times 4)$.

Vectores de Interrupciones



Siempre que se ejecuta una instrucción de interrupción por programación:

1. Se empujan las banderas en la pila.
2. Se ponen en cero los bits de bandera IF y TF.
3. Se empuja CS en la pila.
4. Se busca en el vector el nuevo valor para CS.
5. Se empuja IP a la pila.
6. Se busca en el vector el nuevo valor para IP.
7. Se salta a la nueva dirección localizada en CS e IP.

La instrucción INT se realiza como un CALL lejana excepto que no sólo empuja a la pila CS e IP, sino que también empuja a las banderas a la pila.

Instrucción INT es una combinación de las instrucciones PUSHF y CALL lejana.

Vectores de Interrupciones



Es importante hacer notar que cuando se ejecuta la instrucción INT, se pone en cero la bandera de interrupción (IF), la cual controla las interrupciones de circuitería externa en la terminal de entrada INTR (requisición de interrupción).

Cuando IF es igual a cero, el microprocesador deshabilita la terminal INTR, y cuando IF es igual a uno, el microprocesador habilita la terminal INTR.

Uso común de Interrupciones



Las interrupciones por programación son más comúnmente usadas para llamar subrutinas del sistema.

Las subrutinas del sistema son comunes a todo el sistema y programas de aplicación.

Las interrupciones por lo general controlan la impresora, el video, y los manejadores de discos por nombrar algunos recursos.

Retorno de Interrupciones (IRET)



La instrucción de retorno de interrupción (IRET) es usada solo con subrutinas de servicio de interrupción generadas por programación o circuitería. Diferente a la instrucción de retorno simple (RET), la instrucción IRET:

- 1) Saca un dato de la pila para IP.
- 2) Saca un dato de la pila para CS.
- 3) Saca un dato de la pila para registro de banderas.

La instrucción IRET desarrolla la misma tarea que las instrucciones POPF y RET.

Siempre que se ejecute una instrucción IRET se obtiene el contenido de IF y TF de la pila. Esto es importante por que se conserva el estado de esos bits de bandera. Si los interruptores fueron habilitados antes de una subrutina de servicio de interrupción, son automáticamente rehabilitados por la instrucción IRET por que esta restablece el registro de bandera.

Ejemplo C con ASM

```
✓ [25] !nasm -f elf suma.asm
0s

✓ [19] !gcc -m32 -c main.c
0s

✓ [26] !gcc -m32 suma.o main.o -o ejemploOAC
0s

✓ 3s !./ejemploOAC

Ingrese el primer numero: 8
Ingrese el segundo numero: 7
La suma de 8 y 7 es: 15
```

```
main.c X suma.asm
1 #include <stdio.h>
2
3 // Declaración de la subrutina en ensamblador externa
4 extern int suma_asm(int a, int b);
5
6 int main() {
7     int num1, num2, res;
8
9     // Solicitar dos números al usuario
10    printf("Ingrese el primer numero: ");
11    scanf("%d", &num1);
12
13    printf("Ingrese el segundo numero: ");
14    scanf("%d", &num2);
15
16    // Llamar a la subrutina en ensamblador
17    //para realizar la suma
18    res = suma_asm(num1, num2);
19
20    // Imprimir el resultado
21    printf("La suma de %d y %d es: %d\n", num1, num2, res);
22
23    return 0;
24 }
```

```
main.c suma.asm X
1 section .data
2 section .bss
3 section .text
4 global suma_asm
5
6 suma_asm:
7     push ebp
8     mov ebp, esp
9     ; Parámetros de entrada:
10    ; [ebp+8] contiene el primer número
11    ; [ebp+12] contiene el segundo número
12
13    mov eax, [ebp+8] ; Cargar el primer número en eax
14    add eax, [ebp+12] ; Sumar el segundo número a eax
15
16    ; Parámetros de salida:
17    ; eax contiene el resultado de la suma
18
19    mov esp, ebp
20    pop ebp
21    ret
```